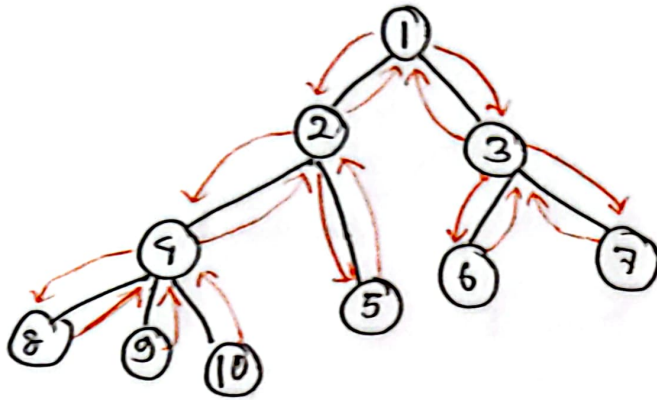


Euler tree:



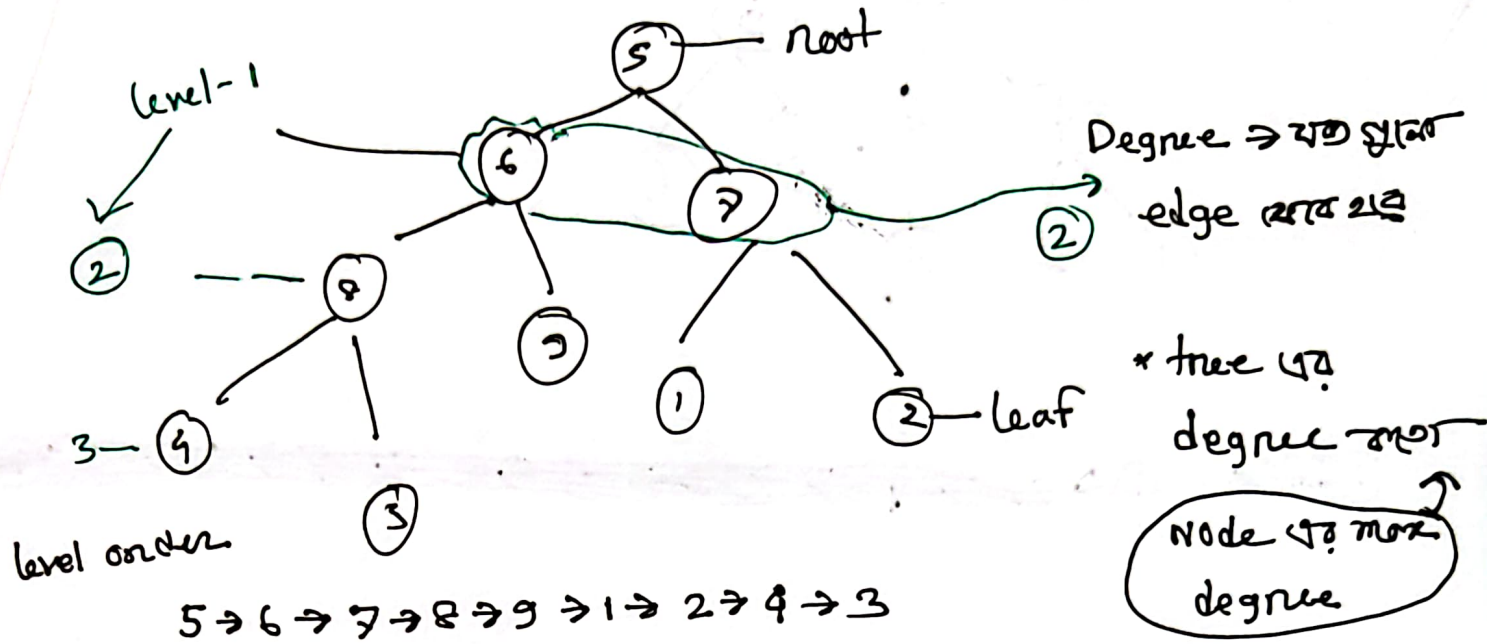
1, 2, 4, 8, 4, 9, 4, 10, 4, 2, 5, 2, 1, 3, 6, 3, 7, 3, 1

Proper Binary tree: children must be 0 or 2 (m-ary tree)

how → graph

Date:

2	8	0	7	2	0	2	3
---	---	---	---	---	---	---	---



ID → left → root → right

Pre → root → left → right

Post → left → right → root

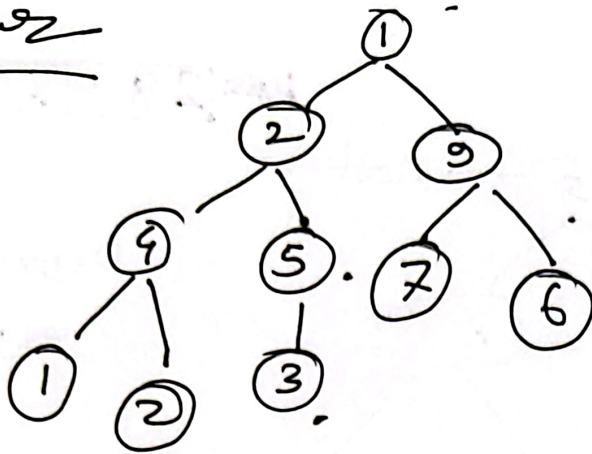
Level → print

[[(0)]]

Glinta[®]

Linagliptin 5mg Tablet

In order



~~1 → 1 → 2 → 4 → 2 → 3 → 5 → 1 → 7 → 9 → 6~~

In order → 1 → 4 → 2 → 2 → 3 → 5 → 1 → 7 → 9 → 6

pre → 1 → 2 → 4 → 1 → 2 → 5 → 3 → 9 → 7 → 6

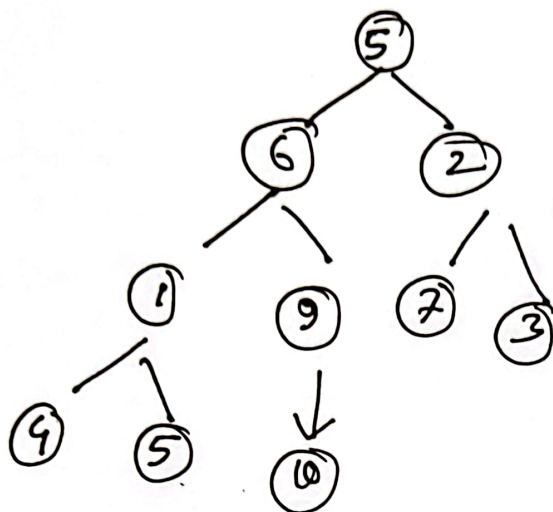


ଆମ root $\rightarrow$ ମୂଳ ---

root $\rightarrow$ left $\rightarrow$ right

Date:

--	--	--	--	--	--	--



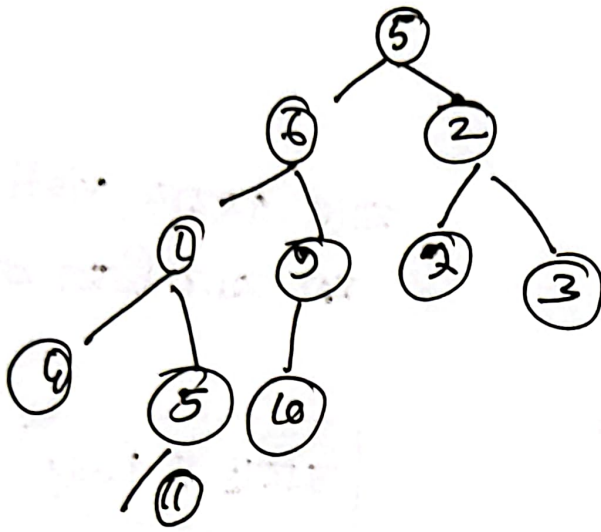
ଯାହା ଏହାଙ୍କ ପିତାଙ୍କ ଦ୍ଵାରା
ପ୍ରାପ୍ତ ହୋଇଥିବା କାରଣ

ହାଲୁକା Easy

5 $\rightarrow$ 6 $\rightarrow$ 1 $\rightarrow$ 4 $\rightarrow$ 5 $\rightarrow$ 9 $\rightarrow$ 10 $\rightarrow$ 2 $\rightarrow$ 7 $\rightarrow$ 3

Post $\Rightarrow$ root ko aakhira (last) visit karna

left $\rightarrow$ right $\rightarrow$ root



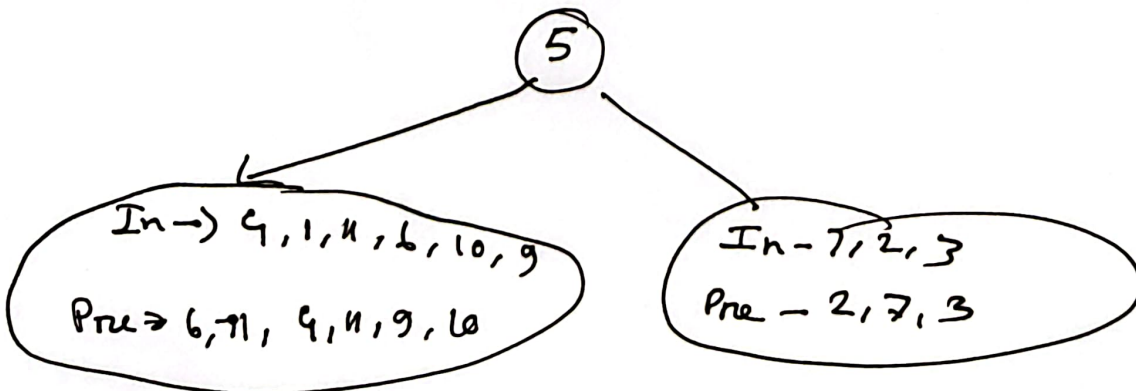
4 $\rightarrow$ 5 $\rightarrow$ 1 $\rightarrow$ 10 $\rightarrow$ 9 $\rightarrow$ 6 $\rightarrow$ 7 $\rightarrow$ 3 $\rightarrow$ 2 $\rightarrow$ 5

220r Traverse karke samajhna $\rightarrow$
in must karna

Tree generate from two traverse

In - 4 $\rightarrow$ 1 $\rightarrow$ 6 $\rightarrow$ 10 $\rightarrow$ 9 $\rightarrow$ 5 $\rightarrow$ 7 $\rightarrow$ 2 $\rightarrow$ 3

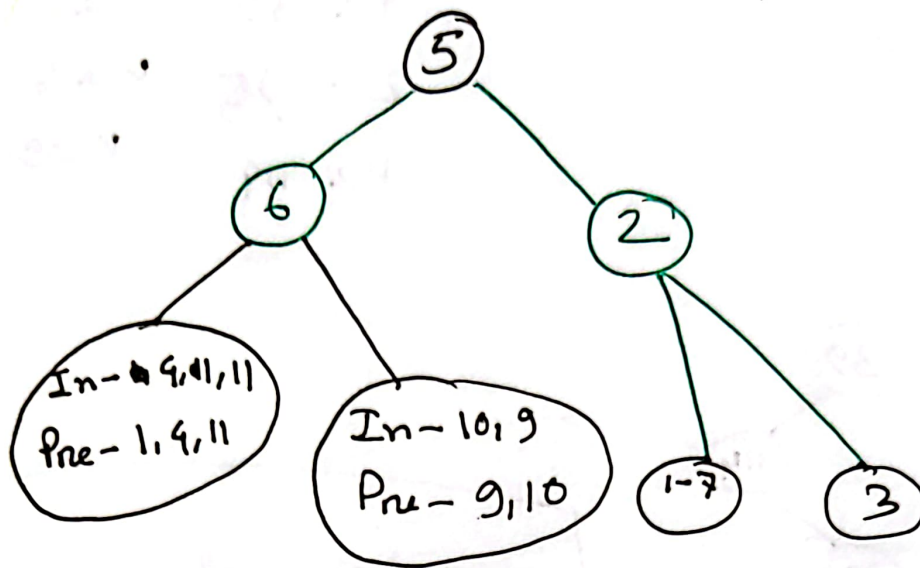
Pre $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 1 $\rightarrow$ 4 $\rightarrow$ 9 $\rightarrow$ 10 $\rightarrow$ 2 $\rightarrow$ 7 $\rightarrow$ 3



Post → 4, 11, 10

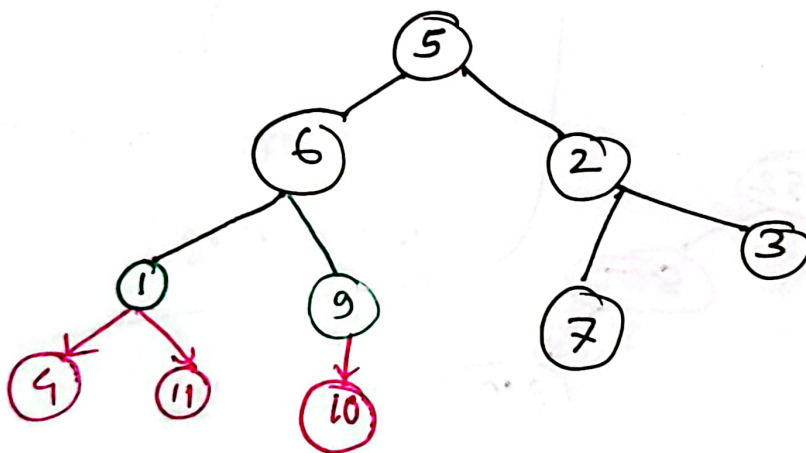
Date:

--	--	--	--	--	--

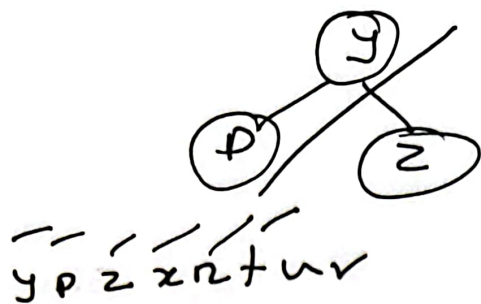


TS Some 2/2
1 and 11/2
2/2

⑧ Post, In tree



① 3-10 tower 12/11 12/11 12/11 12/11
find 12/11 12/11 simulation only
Tower of Hanoi



x = 136

y = 39

z = 75

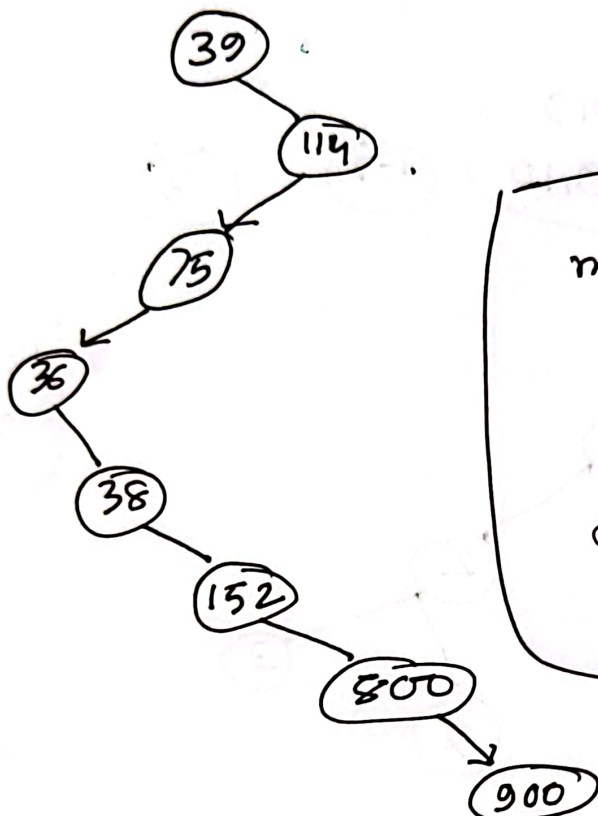
p = 114

r = 38

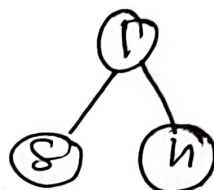
t = 152

u = 800

v = 900



max heap call - 20/12

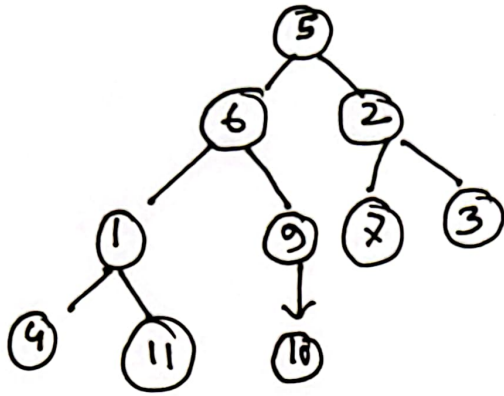


ଏହି କ୍ରମଟି ସଠିକ୍ ନାହିଁ

swap କରାଯାଉ?

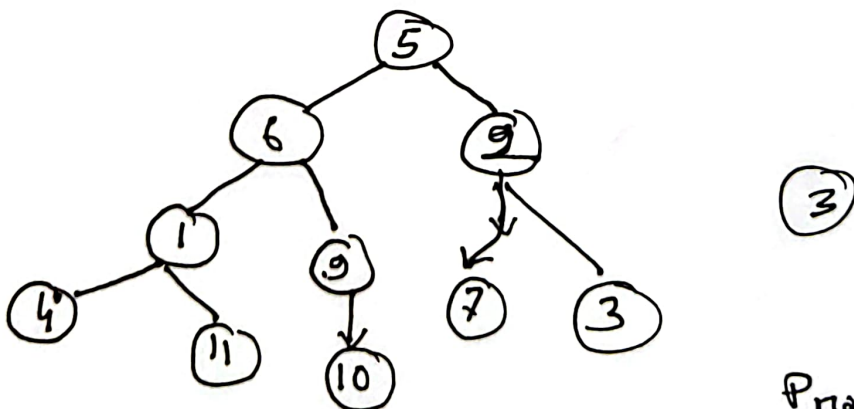
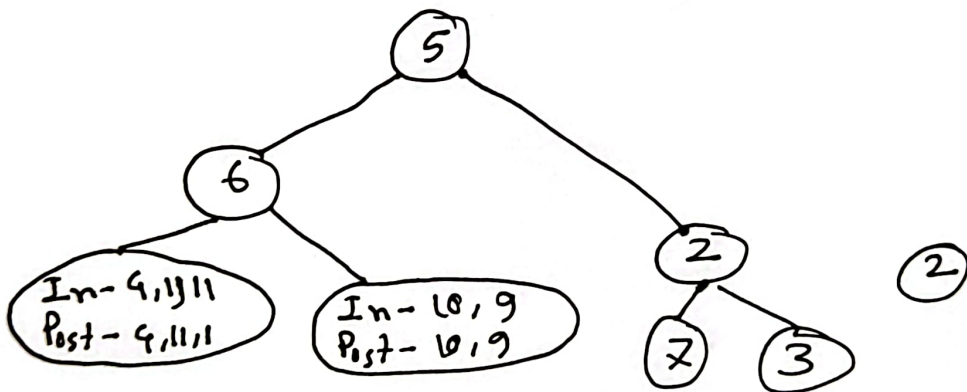
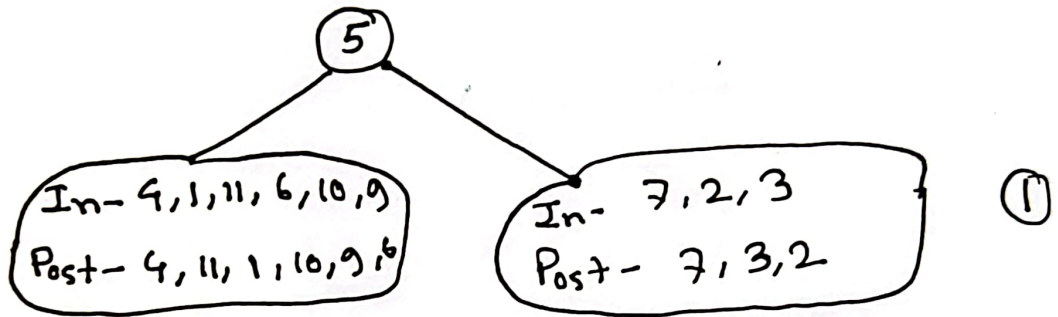
22 32 1
 15 21
 8 9 0
 9 0 0

Date:



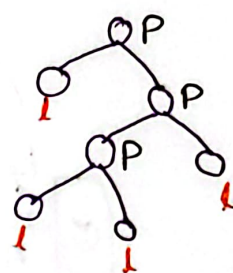
In - 4, 1, 11, 6, 10, 9, 5 → 7, 2, 3

Post - 4, 11, 1, 10, 9, 6, 7, 3, 2, 5



Proved

n: A proper binary tree with n internal nodes, has $n+1$ external nodes (leaves)



Internal = 3
External = 4

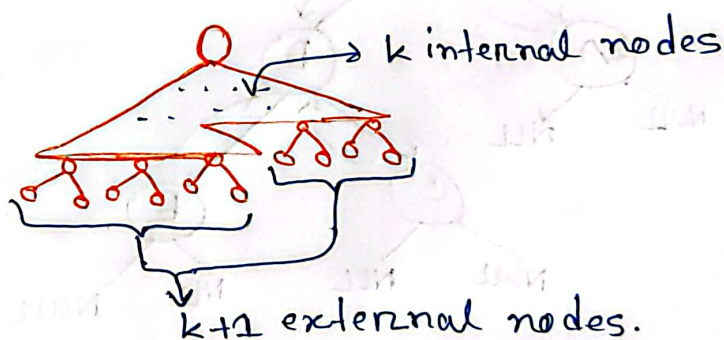
Basis Step:

$P(0)$: A proper binary tree with 0 internal nodes, has 0+1 external nodes.

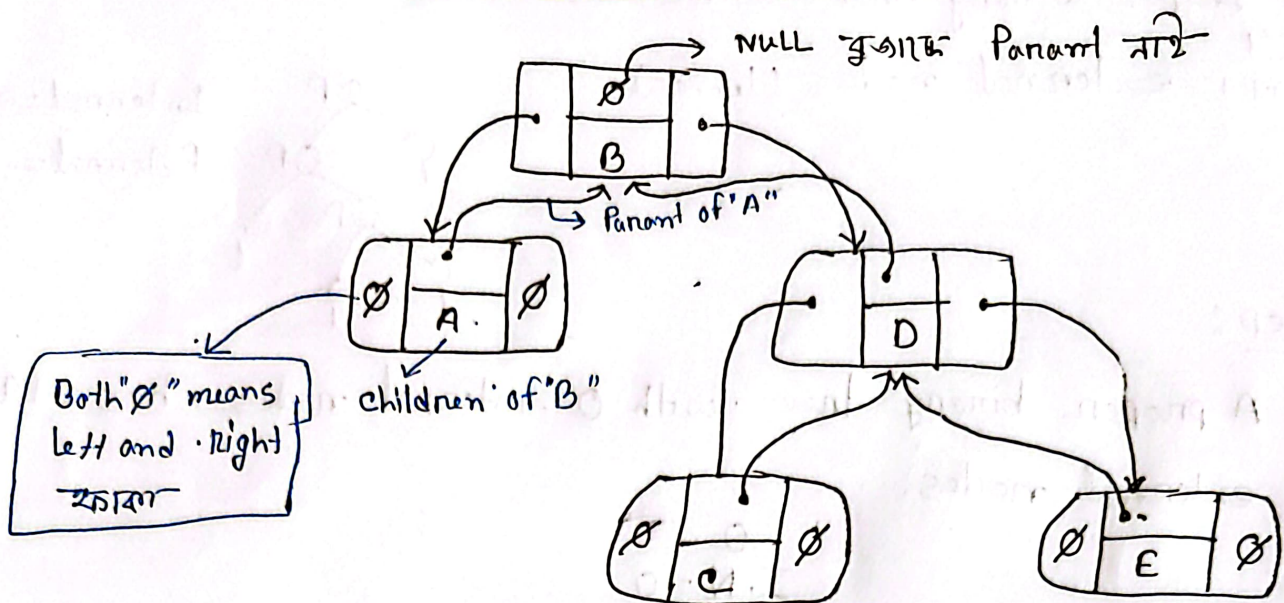
$$\begin{aligned} I.N &= 0 \\ E.N &= 1 \end{aligned}$$

Inductive Step: let us assume $P(k)$ to be true.

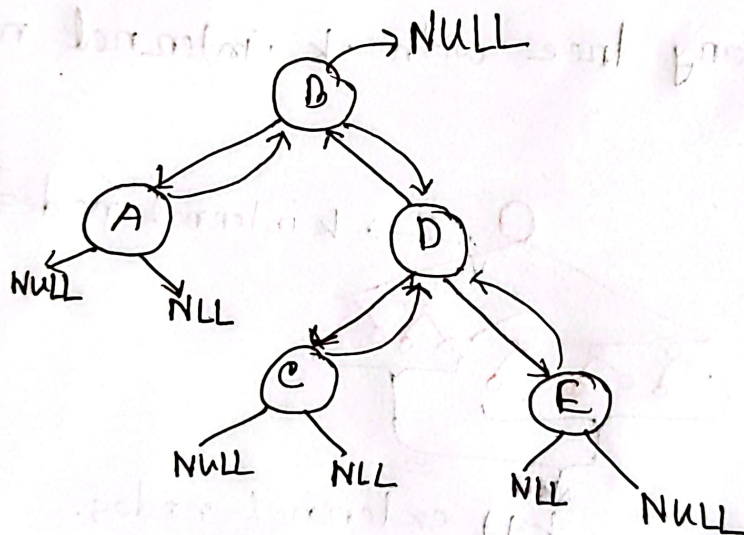
$P(k)$ A proper binary tree with k internal nodes, has $k+1$ external nodes.



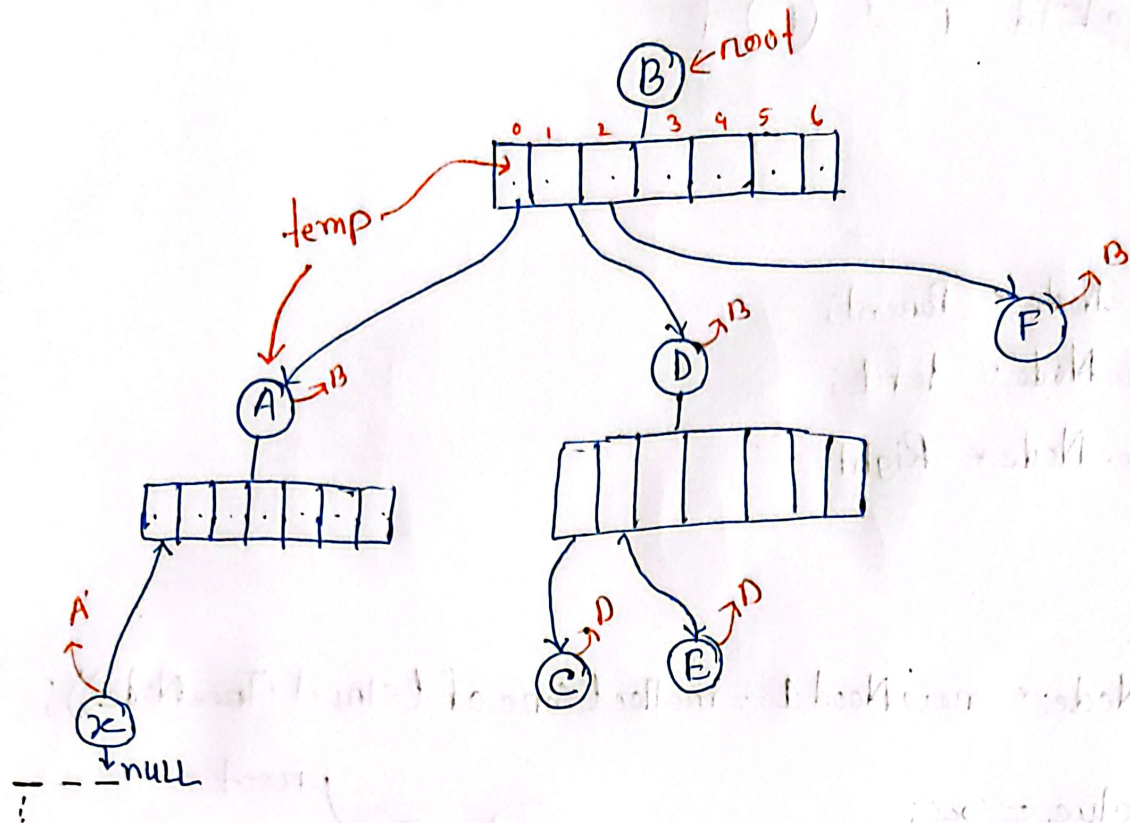
Implementing Binary Tree



OR



Write a code to add children 'x' and 'z' to (A) and Y to (E)



```

① void add(x) {
    new_node 1 = malloc(...);
    new_node 1 -> value = 'x';
    new_node 1 -> Parent = NULL;
    new_node 1 -> children = NULL;
    temp = root -> childCont -> ChildArray [0];
    new_node 1 -> Parent = temp;

```

```

② -> temp -> childrenCont = malloc (size of (struct container));
    temp -> ChildContainer -> ChildArray [0] = new_node 1;
    temp -> child Con -> ChildArray ++;
}

```

```

③ new_node 2 = malloc(-);
    new_node 2 -> value = 'z';
    new_node 2 -> Parent = temp;
    new_node 2 -> childCont = NULL;
    temp -> childCont -> ChildArray [ ] = new_node 2;
    new_node 2 -> childContainer = NULL;

```

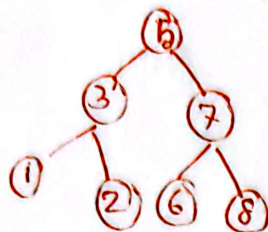
] = new_node 2;

Binary Search Tree

Time complexity of Insert, delete, search, min, max in BST is $\rightarrow O(n)$

Search
Minimum
Maximum
Predecessor
Successor
Insert
Delete

1 2 3 4 5 6 7 8



Balanced tree :

Tree of height $n \rightarrow h \approx \sqrt{n}$ then,

leaf of height/depth $\rightarrow h-1$

Time complexity of a balanced tree $\rightarrow h = O(\log n)$
 $\downarrow$
 $O(h)$

Search

Code

```
node *TSearch (node *root, int key)
```

```
{ if (root == NULL)
```

```
    return root NULL;
```

```
    else if (key < root->key) {
```

```
        key = root->left;
```

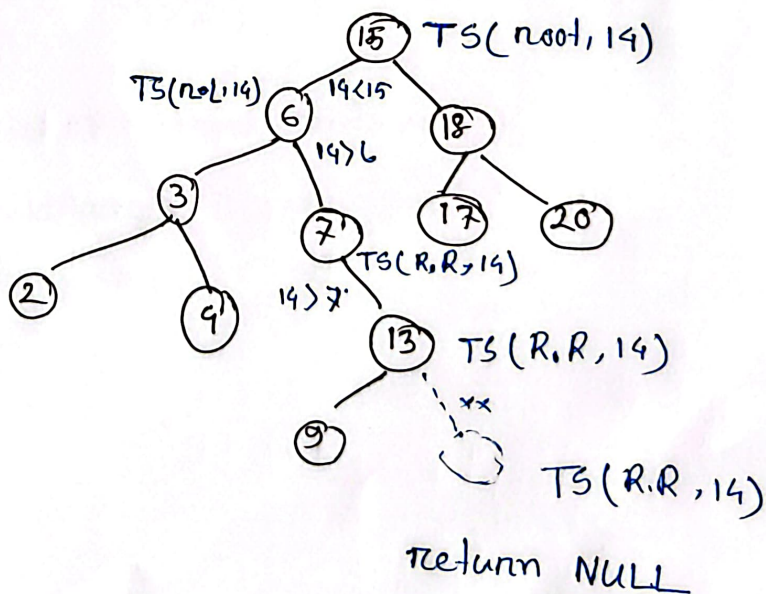
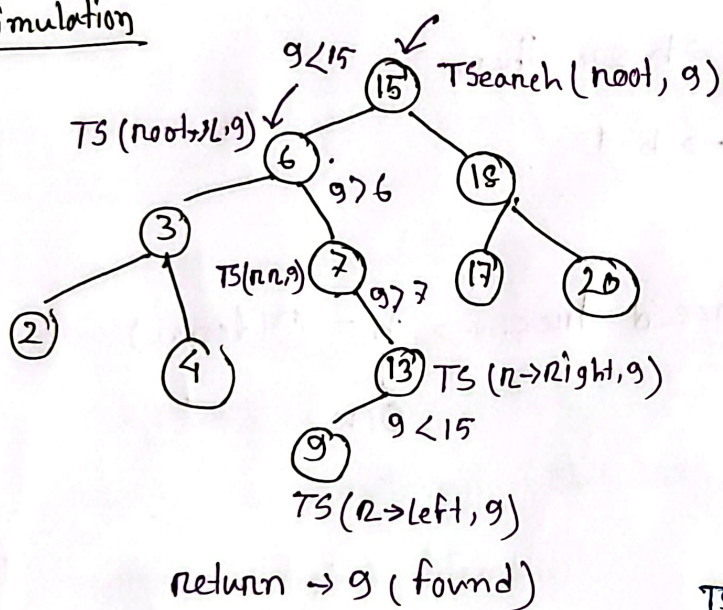
```
    }
```

```
    else {
```

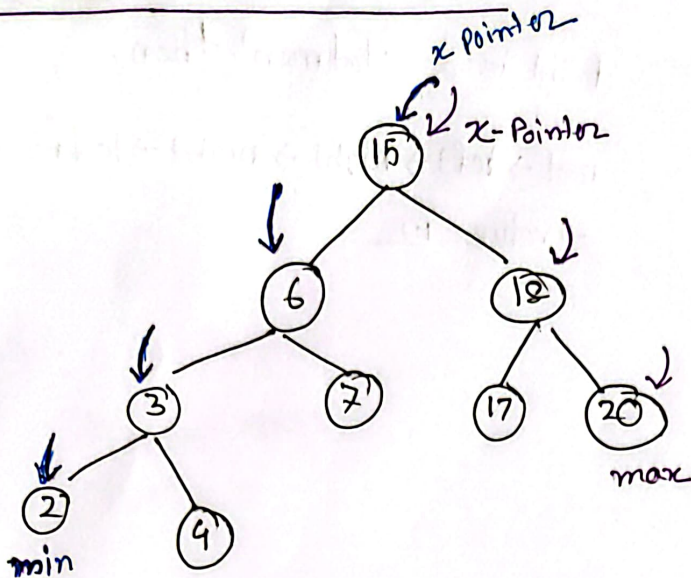
```
        key = root->right;
```

```
    }
```

Simulation



Minimum and Maximum:



Tree-Min(x)

```

{ while (x.left != NULL)
{
    x = x.left;
}
return x;
}

```

Tree-Max(x)

```

{ while (x.right != NULL)
{
    x = x.right;
}
return x;
}

```

Successor and Predecessor

Successor

Case 1: যদি node এর right child থাকে

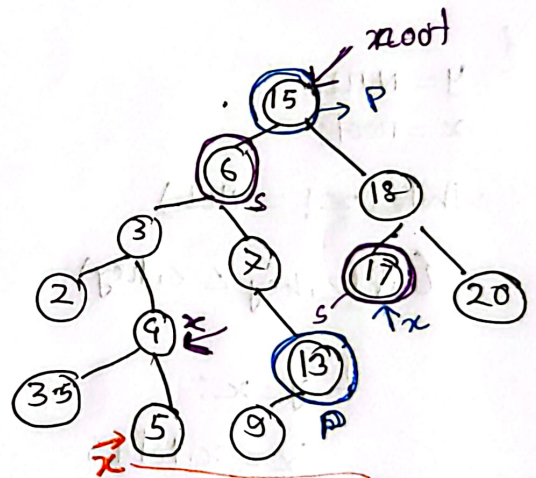
Successor(root) $\rightarrow$ 17

* এক্ষেত্রে যদি ডান দিকের সর্বোচ্চ successor

Case 2: যদি Right child না থাকে

Successor(x) $\rightarrow$ 6

* Nearest ancestor / এক্ষেত্রে Parent এর Successor



Tree-Successor(x)

```

{ if (x.right != NULL)
{
    return Tree-Min(x.right);
}
y = x.P;
while (y != NULL && x == y.right)
{
    x = y;
    y = y.P;
}
return y;
}

```

Predecessor

Case 1: যদি node এর left child থাকে

Predecessor(root) $\rightarrow$ 13

* এক্ষেত্রে যদি বাঁ দিকের সর্বোচ্চ max value

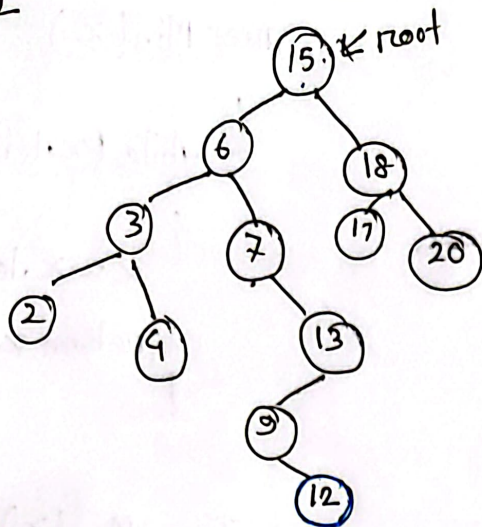
Case 2: যদি left child না থাকে

Predecessor(x) $\rightarrow$ 15

* Nearest ancestor / এক্ষেত্রে Parent এর Predecessor.
immediate

Insert

z=12

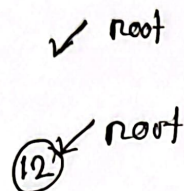


if it is a statement then,

root → left → right → right → left → right
→ value = 12

Insert in empty tree

z=12



void Insert(z)

```
{
  y = NULL;
  x = root;
```

```
while(x != NULL)
```

```
{ if(z.key < x.key)
```

```
{
  y = x;
  x = x.left;
```

```
}
```

```
else
```

```
{
  y = x;
  x = x.right;
```

```
}
```

```
if y == NULL
```

```
root = z;
```

```
if (z.key > y.key)
```

```
  y.right = z
```

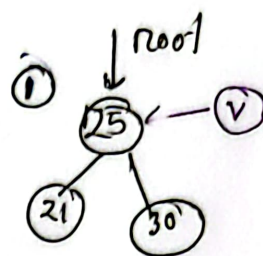
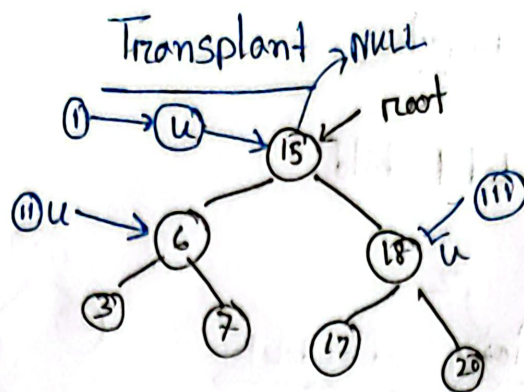
```
  z.p = y
```

```
else
```

```
  y.left = z
```

```
  z.p = y
```





case 1: u is a root

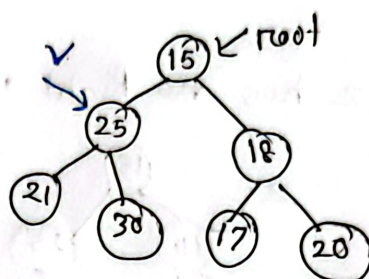
if ($u.p == \text{NULL}$)

$\text{root} = v$

case 2: u is a left child

if ($u == u.p.\text{left}$)

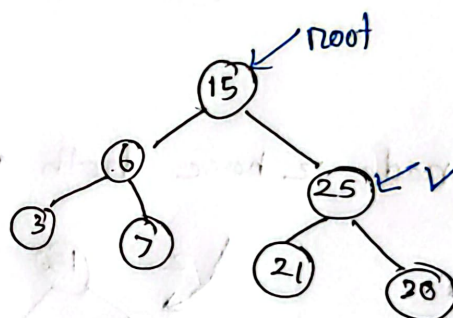
$u.p.\text{left} = v$



case 3: u is a Right child

if ($u == u.p.\text{right}$)

$u.p.\text{right} = v$



Pseudocode

Transplant(u, v)

if ($u.p == \text{NULL}$)

$\text{root} = v$

else if ($u == u.p.\text{left}$)

$u.p.\text{left} = v$

else

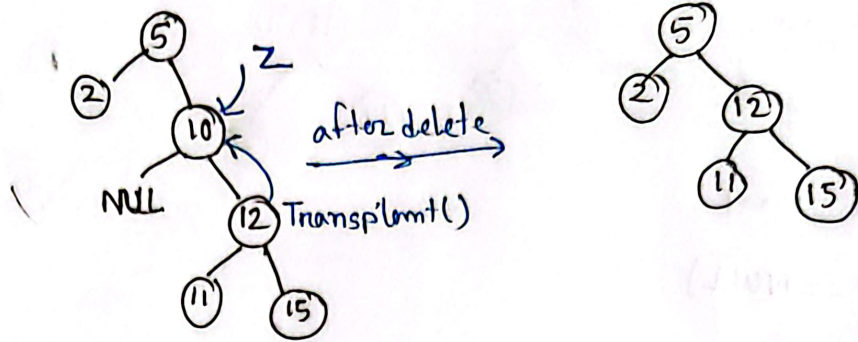
$u.p.\text{right} = v$

if ($v.p == \text{NULL}$)

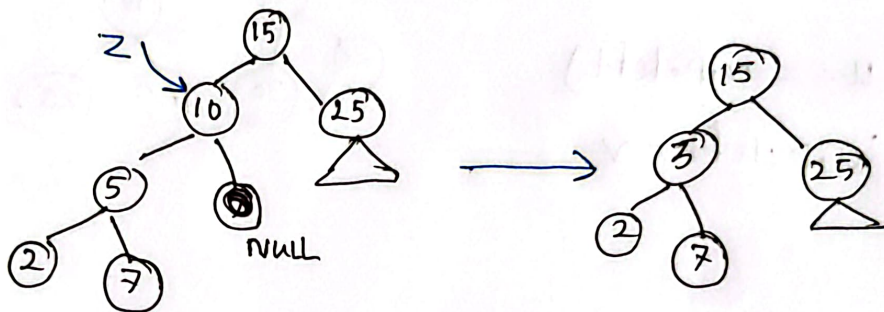
$v.p = u.p$

Delete

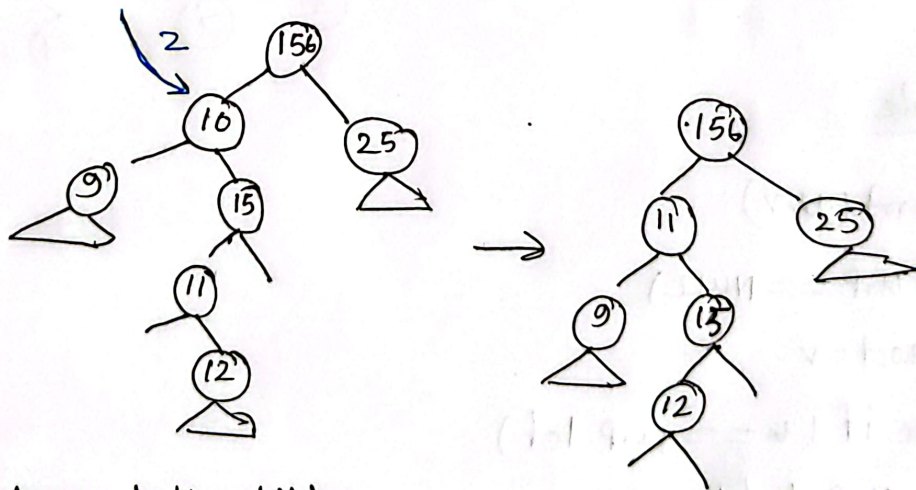
case 1: node z has no left child



case 2: node z has no Right child



case 3: node z has both child



case 4: node z has both child

